



Probabilistic Machine Learning

23. Graph Embeddings

박준수 / 2026.05.28



Computational Data Science LAB



CONTENTS

1. Introduction & Motivation
 2. GraphEDM Framework
 3. Shallow Graph Embeddings
 4. Graph Neural Networks
 5. Deep Graph Embeddings
- 
- 

01 | Introduction & Motivation

Learning settings and non-Euclidean embedding

- Graph Representation Learning (GRL)
 - ✓ 그래프 데이터를 저차원 연속 벡터(embedding)로 표현
 - ✓ 비지도 GRL: 그래프 구조 자체를 보존하도록 학습
 - ✓ 지도 GRL: 노드/엣지/그래프 라벨 등 downstream task 동시 최적화
- Transductive vs Inductive 학습 환경
 - ✓ Transductive: 학습 시 본 동일 그래프에서 예측 (예: 소셜 네트워크 사용자 분류)
 - ✓ Inductive: 학습 시 보지 않은 새 그래프에 일반화 (예: 분자 분류)
- 비유클리드 임베딩 공간 (Non-Euclidean Embedding)
 - ✓ Euclidean 공간: 내적 · 거리 계산 단순, 그러나 트리 · 계층 구조를 왜곡 없이 표현하기 어려움
 - ✓ Hyperbolic(쌍곡) · Spherical(구면) 공간: 데이터 구조에 맞춰 임베딩 공간 선택 → 왜곡 최소화

02 | GraphEDM Framework

Encoder-decoder formulation and method taxonomy

- 입력 / 출력 / Encoder-Decoder

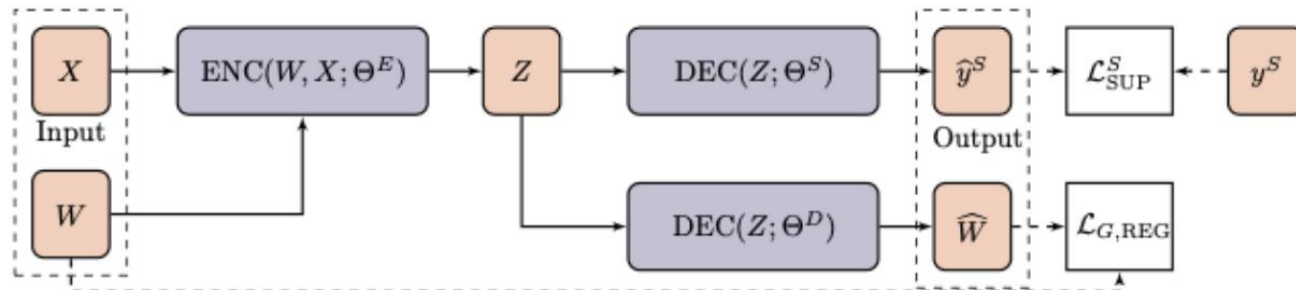
- ✓ 입력: $\mathbf{W} \in \mathbb{R}^{N \times N}$, (선택) $\mathbf{X} \in \mathbb{R}^{N \times D}$; 지도 학습 시 노드(N) / 엣지(E) / 그래프(G) 라벨 추가
- ✓ Encoder: $\mathbf{Z} = \text{ENC}(\mathbf{W}, \mathbf{X}; \theta^E) \in \mathbb{R}^{N \times L}$ — 노드 임베딩 행렬
- ✓ Decoder: $\hat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \theta^D)$ — 구조 재구성 / $\hat{y}^S = \text{DEC}(\mathbf{Z}; \theta^S)$ — 라벨 예측

- 통합 손실 함수

- ✓ $\mathcal{L} = \alpha \mathcal{L}_{\text{SUP}}^S(y^S, \hat{y}^S; \theta) + \beta \mathcal{L}_{G, \text{RECON}}(\mathbf{W}, \hat{\mathbf{W}}; \theta) + \gamma \mathcal{L}_{\text{REG}}(\theta)$
- ✓ 각 항의 의미: 지도 손실 / 그래프 구조 재구성 손실 / 파라미터 prior(가중치 정규화)

- Hyperparameter (α, β, γ) 에 따른 분류

- ✓ $\alpha = 0 \Rightarrow$ Unsupervised (구조만으로 임베딩 학습), $\alpha \neq 0 \Rightarrow$ Supervised / Semi-supervised (라벨 활용)
- ✓ $\beta > 0 \Rightarrow$ 그래프 구조 재구성 항을 함께 최적화 $\rightarrow$ Encoder/Decoder 선택에 따라 MF, DeepWalk, GCN, GAT 등 통합 표현



03 | Shallow Graph Embeddings

Unsupervised embeddings — Euclidean & Non-Euclidean methods

- Shallow embedding 의 정의

- ✓ Encoder = lookup table: $\mathbf{Z} = \text{ENC}(\theta^E) \triangleq \theta^E$, $\theta^E \in \mathbb{R}^{N \times L}$, 각 노드 ID를 학습 가능한 $\mathbf{Z}_i$ 에 직접 매핑(Transductive)
- ✓ 두 가지 계열: Distance-based: $\hat{W}_{ij} = d_2(\mathbf{Z}_i, \mathbf{Z}_j)$ / Outer product-based: $\hat{\mathbf{W}} = \mathbf{Z}\mathbf{Z}^\top$
- ✓ 비지도 손실: $\mathcal{L}_{G, \text{RECON}}(\mathbf{W}, \hat{\mathbf{W}}; \theta) = d_1(s(\mathbf{W}), \hat{\mathbf{W}})$, $s(\mathbf{W})$: 그래프 관계 행렬 변환 ($\mathbf{W}, \mathbf{W}^2, 1 - \mathbf{W} \dots$)

- 유클리드법 (Distance-based Euclidean)

- ✓ MDS: $\hat{W}_{ij} = \|\mathbf{Z}_i - \mathbf{Z}_j\|$, $d_1(s(\mathbf{W}), \hat{\mathbf{W}}) = \|s(\mathbf{W}) - \hat{\mathbf{W}}\|_F^2 \rightarrow$ 그래프에서 정의한 노드 간 거리/비유사도 행렬을 저차원 임베딩 거리로 보존
- ✓ Laplacian Eigenmaps: 목적 - 연결된 노드 쌍은 임베딩에서도 가깝게 $\rightarrow \min_{\mathbf{Z} \in \mathbb{R}^{|V| \times d}} \text{tr}(\mathbf{Z}^\top \mathbf{L} \mathbf{Z})$
- ✓ 단, $\text{tr}(\mathbf{Z}^\top \mathbf{L} \mathbf{Z}) = \frac{1}{2} \sum W_{ij} \|\mathbf{Z}_i - \mathbf{Z}_j\|^2$ s.t. $\mathbf{Z}^\top \mathbf{D} \mathbf{Z} = \mathbf{I}$: 모든 노드가 한 점에 뭉치는 자명한 해 방지 ($\mathbf{L} = \mathbf{D} - \mathbf{W}$)

- 비유클리드법 (Distance-based Hyperbolic)

- ✓ 동기: 트리 · 계층 구조를 유클리드에 임베딩 시 왜곡 큼 $\rightarrow$ Hyperbolic = '트리의 연속적 일반화'
- ✓ Poincaré: $d_2(\mathbf{Z}_i, \mathbf{Z}_j) = \text{arcosh} \left(1 + 2 \frac{\|\mathbf{Z}_i - \mathbf{Z}_j\|_2^2}{(1 - \|\mathbf{Z}_i\|_2^2)(1 - \|\mathbf{Z}_j\|_2^2)} \right)$, Riemannian 최적화

03 | Shallow Graph Embeddings

Outer product-based: Matrix Factorization

- Matrix Factorization (MF) 의 핵심

- ✓ $\hat{\mathbf{W}} = \text{DEC}(\mathbf{Z}; \theta^D) = \mathbf{Z}\mathbf{Z}^\top \rightarrow \text{low-rank 분해}$

- ✓ $s(\mathbf{W}) \in \{\mathbf{W}, \mathbf{L}, \text{Katz}, \text{Adamic/Adar} \dots\}$

- ✓ $\mathcal{L}_{G, \text{RECON}} = \|\mathbf{s}(\mathbf{W}) - \hat{\mathbf{W}}\|_F^2$

- Graph Factorization (GF)

- ✓ 실제 존재하는 엣지에 대해서만 재구성 손실 계산 $\rightarrow$ 희소 연산, 빠름

- ✓ $\mathcal{L}_{G, \text{RECON}} = \sum_{(v_i, v_j) \in E} (W_{ij} - \hat{W}_{ij})^2 = \|\mathbf{A} \odot (\mathbf{W} - \hat{\mathbf{W}})\|_F^2$, 복잡도 $O(M)$

- GraRep — 비대칭 · k-hop 일반화

- ✓ 노드마다 source $\mathbf{Z}^s$, target $\mathbf{Z}^t$ 두 개 학습

- ✓ $\hat{\mathbf{W}}^{(k)} = \mathbf{Z}^{(k),s} \mathbf{Z}^{(k),t\top}$, $\mathcal{L}^{(k)} = \|\mathbf{D}^{-k} \mathbf{W}^k - \hat{\mathbf{W}}^{(k)}\|_F^2$

- ✓ 그래프 관계 행렬의 거듭제곱으로 k-hop 관계를 만들고, 이를 재구성하는 오차를 최소화하여 k-hop 이웃 구조를 보존

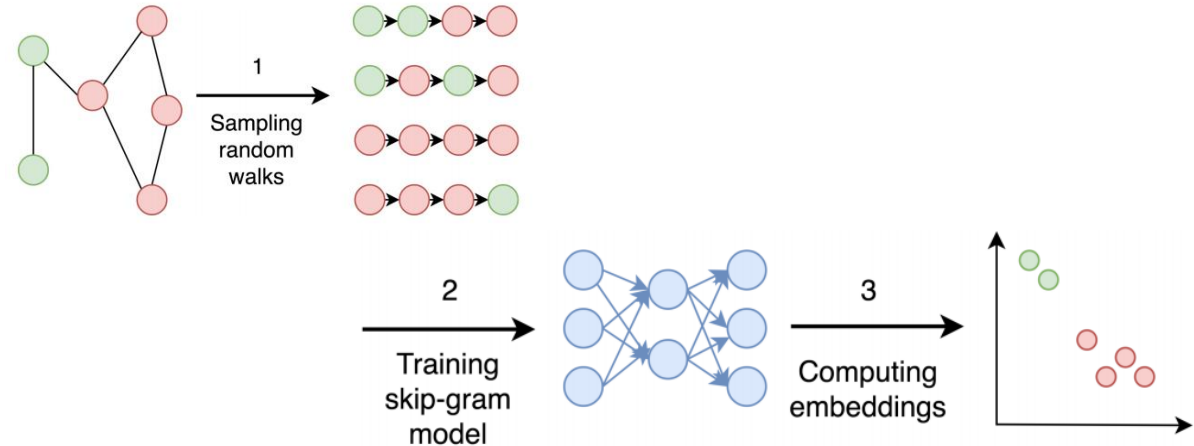
- ✓ 단점: $\mathbf{D}^{-1} \mathbf{W}$ 거듭제곱이 dense 해져 큰 그래프에 적용 어려움

03 | Shallow Graph Embeddings

Skip-gram methods: DeepWalk and node2vec

- Skip-gram 아이디어 (자연어 → 그래프)

- ✓ Skip-gram은 중심 단어를 보고 주변 단어가 함께 등장할 확률을 높이도록 학습하는 방법
- ✓ $\mathcal{L} = -\sum_{-K \leq i \leq K, i \neq 0} \log \mathbb{P}(w_{k-i} | w_k)$
- ✓ 단어 빈도 분포 $\approx$ 그래프 random walk 의 노드 빈도 분포



- DeepWalk

- ✓ Truncated random walk → 시퀀스 생성 → skip-gram 학습
- ✓ GraphEDM: $\hat{\mathbf{W}} = \mathbf{Z}\mathbf{Z}^T, s(\mathbf{W}) = \mathbb{E}_q[(\mathbf{D}^{-1}\mathbf{W})^q]$
- ✓ $\mathcal{L}_{G, \text{RECON}}(\mathbf{W}, \hat{\mathbf{W}}; \theta) = \log Z(\mathbf{Z}) - \sum_{v_i, v_j \in V} s(\mathbf{W})_{ij} \hat{W}_{ij}$
- ✓ Random walk에서 자주 함께 등장한 노드 쌍은 임베딩 공간에서도 더 비슷해지도록 학습

- node2vec 및 NetMF

- ✓ node2vec: random walk 의 편향 (p, q) 으로 BFS/DFS-like 탐색 균형
- ✓ NetMF: DeepWalk · LINE · node2vec 를 implicit MF 로 통합 (Skip-gram 학습을 그래프 관계 행렬분해 관점에서 해석)

03 | Shallow Graph Embeddings

Supervised shallow embeddings: Label Propagation & Spreading

- 동기

- ✓ Unsupervised → 별도 분류기 2-step 방식은 downstream 에 비최적
- ✓ 임베딩 학습과 라벨 예측을 결합하는 단일 손실 사용

- Label Propagation (LP)

- ✓ $\hat{y}^N = \text{DEC}(\mathbf{Z}; \theta^C) = \mathbf{Z}$ (decoder 항등함수)
- ✓ $\mathcal{L}_{G,\text{RECON}} = \sum_{i,j} W_{ij} \|y_i^N - \hat{y}_j^N\|_2^2$
- ✓ 라벨된 노드는 고정, 나머지는 인접 노드의 가중 평균으로 반복 갱신 (수렴 시까지)

- Label Spreading (LS) & 지도 손실

- ✓ LS: $\mathcal{L}_{G,\text{RECON}} = \sum_{i,j} W_{ij} \left\| \frac{\hat{y}_i^N}{\sqrt{D_i}} - \frac{\hat{y}_j^N}{\sqrt{D_j}} \right\|_2^2$
- ✓ $\mathcal{L}_{\text{SUP}}^N(y^N, \hat{y}^N; \theta) = \sum_{i|v_i \in V_L} \|y_i^N - \hat{y}_i^N\|_2^2$
- ✓ 라벨도 부드럽게 변동 허용, degree로 정규화된 이웃 가중 평균으로 갱신

04 | Graph Neural Networks

Message Passing GNNs (GNN, GGSNN, MPNN)

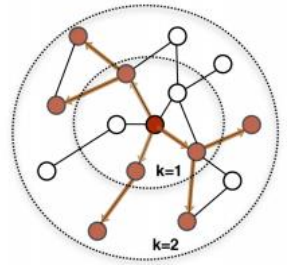
- 기본 GNN 모델
 - ✓ 그래프 임베딩을 '정보 확산(diffusion)' 과정으로 해석
 - ✓ $\mathbf{Z}^{t+1} = \text{ENC}(\mathbf{X}, \mathbf{W}, \mathbf{Z}^t; \theta^E)$, 수렴 후 $\hat{y}^S = \text{DEC}(\mathbf{X}, \mathbf{Z}^T; \theta^S)$
 - ✓ Banach 고정점 정리: 수축사상(contraction) 일 때 유일한 해 존재
- Gated GraphSeq NN (GGSNN)
 - ✓ 고정점 가정 제거 — 고정된 횟수의 step 만 적용, GRU 매핑
- Message Passing NN (MPNN) — 표준 추상화
 - ✓ $\mathbf{m}_i^{\ell+1} = \sum_{j|(v_i, v_j) \in E} f^\ell(\mathbf{H}_i^\ell, \mathbf{H}_j^\ell)$
 - ✓ $\mathbf{H}_i^{\ell+1} = h^\ell(\mathbf{H}_i^\ell, \mathbf{m}_i^{\ell+1}), \mathbf{H}^0 = \mathbf{X}$
 - ✓ ℓ 층 후, 노드는 ℓ -hop 이웃 정보를 hidden state 에 인코딩

04 | Graph Neural Networks

Spectral & Spatial Graph Convolutions

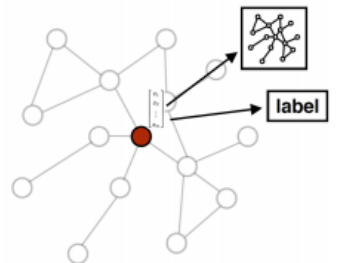
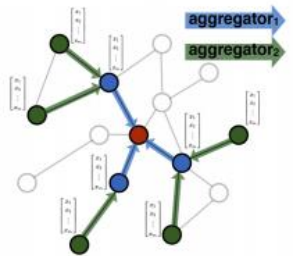
- 스펙트럴(Spectral) 방법

- ✓ 그래프를 주파수 관점에서 해석해 convolution을 수행
- ✓ 이미지에서 FFT를 통해 신호를 주파수 성분으로 나누는 것처럼, 그래프 신호도 부드러운 성분과 거친 성분으로 분해
- ✓ Spectral CNN은 고유분해를 사용하지만, 계산 비용이 크고 새 그래프에 일반화가 어려움
- ✓ GCN은 고유분해 없이 주변 노드 정보를 1차 근사로 집계해 계산을 단순화



- 공간적(Spatial) 방법

- ✓ 노드의 이웃 정보를 직접 모아 현재 노드 표현을 업데이트
- ✓ GraphSAGE는 일부 이웃만 sampling하여 계산량을 줄이고, 새로운 그래프에도 적용 가능
- ✓
$$\mathbf{H}_{:,i}^{\ell+1} = \sigma \left(\theta_1^\ell \mathbf{H}_{:,i}^\ell + \theta_2^\ell \text{AGG}(\{\mathbf{H}_{:,j}^\ell | v_j \in \text{Sample}(\text{nbr}(v_i), q)\}) \right)$$
- ✓ GAT는 이웃마다 중요도를 다르게 주어 더 중요한 이웃 정보를 더 많이 반영



04 | Graph Neural Networks

Non-Euclidean GNNs (HGCN, HGNN)

- 동기

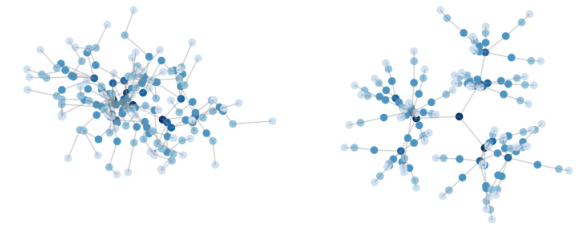
- ✓ Shallow hyperbolic 임베딩 — 표현력 좋지만 inductive 일반화 어려움
- ✓ GNN — inductive 강점, 하지만 기본은 Euclidean
- ✓ 두 장점을 결합 → Non-Euclidean GNN

- Hyperbolic GCN (HGCN) / Hyperbolic GNN (HGNN)

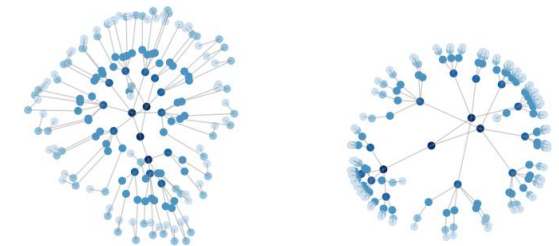
- ✓ Hyperbolic manifold 위에서 convolution 정의
- ✓ 핵심: tangent space (manifold 의 1차 근사) 활용
- ✓ ① hyperbolic → tangent space ② tangent 에서 conv ③ tangent → hyperbolic

- 결과

- ✓ 트리 등 hierarchical 구조에서 Euclidean GCN보다 낮은 거리 왜곡을 보이는 경우가 많음
- ✓ 노드 색(=depth) 시각화 시 임베딩 공간에 자연스러운 계층 형성



(a) GCN layers.



(b) HGCN layers.

05 | Deep Graph Embeddings

Unsupervised Deep Graph Embeddings

- 재구성 기반

- ✓ 임베딩을 이용해 원래 그래프의 연결 구조를 다시 복원하도록 학습
- ✓ SDNE는 Auto-encoder로 그래프 관계 행렬 정보를 압축하고 복원
- ✓ GAE는 GCN으로 노드 임베딩을 만든 뒤, 두 노드 사이의 연결 가능성을 예측
- ✓ Encoder: $\mathbf{Z} = GCN(\mathbf{X}, \mathbf{W}; \theta^E)$, Decoder: $\hat{\mathbf{W}} = \sigma(\mathbf{Z}\mathbf{Z}^\top)$
- ✓ VGAE는 GAE에 확률적 latent variable을 추가한 모델
- ✓ Graphite는 decoder에 더 복잡한 모델을 도입하여 그래프 복원 성능을 높인 방법

- 대조 기반

- ✓ 실제 그래프와 오염(corrupted) 그래프를 구분하는 self-supervised 방식으로 노드 임베딩 학습
- ✓ 실제그래프: $\mathbf{Z} = ENC(\mathbf{X}, \mathbf{W}; \theta^E) \in \mathbb{R}^{N \times L}$, 오염 그래프: $\mathbf{Z}^- = ENC(\mathbf{X}^-, \mathbf{W}^-; \theta^E) \in \mathbb{R}^{N^- \times L}$
- ✓ DGI: 실제 그래프의 노드 임베딩 $\mathbf{Z}$ 로부터 전체 그래프 요약 벡터 $s = READOUT(\mathbf{Z})$ 를 생성
- ✓ READOUT: 여러 노드 임베딩을 평균/합산하여 그래프 전체를 대표하는 요약 벡터 s 생성
- ✓ 실제 노드-그래프 쌍 $(\mathbf{z}_i, s)$ 는 높은 점수, 오염 노드-그래프 쌍 $(\mathbf{z}_i^-, s)$ 는 낮은 점수를 갖도록 학습

05 | Deep Graph Embeddings

Semi-supervised Deep Graph Embedding

- Semi-supervised embedding 의 동기

- ✓ 일부 노드에만 라벨이 있는 상황에서 그래프 임베딩을 학습
- ✓ 그래프 구조 정보와 라벨 정보를 함께 활용
- ✓ 구조 정보는 연결된 노드를 가깝게 만들고, 라벨 정보는 분류에 유용한 임베딩을 만들도록 도움

- SemiEmb

- ✓ $\hat{W}_{ij} = \| \mathbf{z}_i - \mathbf{z}_j \|^2$
- ✓ MLP encoder로 노드 임베딩 $\mathbf{z}_i$ 를 생성
- ✓ 연결된 노드들이 임베딩 공간에서도 가깝게 위치하도록 학습
- ✓ Label propagation 정규화를 사용해 일부 라벨 정보를 주변 노드로 확산
- ✓ 학습 후에는 반복 계산 없이, 노드 특징을 MLP에 넣어 바로 예측

- Planetoid

- ✓ $\mathbf{Z} = [\mathbf{Z}^c \parallel \mathbf{Z}^F] = \text{ENC}(\mathbf{X}; \theta^E)$
- ✓ $\mathbf{Z}^c$: 그래프 구조 기반 임베딩
- ✓ $\mathbf{Z}^F$: 노드 특징 기반 임베딩
- ✓ 구조 정보와 특징 정보를 결합해 노드 표현을 학습
- ✓ $\mathcal{L}_{G, \text{RECON}} = -\mathbb{E}_{(i,j,\gamma)} \log \sigma(\gamma \cdot \hat{W}_{ij}), \gamma \in \{-1, +1\}$
- ✓ 그래프 구조 보존 손실과 라벨 예측 손실을 함께 최적화

Q&A

감사합니다.